How to Read a Source Card

Every source in column B is one card, and every card has the same six parts in the same order. This edition adds the technical detail: the markup behind each part, and how the previews get their content without the page ever reaching out to the internet.

The example used throughout is card 15 on atomic-secrets.online:

> 15 · COURT RECORD
>
> **Estate of Peter Vogel, also known as Robert Lance Vogel**
>
> *Kings County Surrogate's Court, Brooklyn · file 2023-2443*
>
> The estate file opened in April 2023: an administration proceeding, with no will, giving the date of death as December 27, 2022. Search by name in Kings County to see the file history.
>
> **Why it is cited:** the only public record of Vogel's death that has been found.
>
> → websurrogates.nycourts.gov · Kings County, file 2023-2443

1. Number and kind

```
15 · Court record
```

The number is the card's ID. Every small red numeral in columns A and C points to a card by that number; clicking one opens column B, scrolls to the card and flashes its border. The kind is a short label in small capitals (Court record, Book, Newspaper, Government report, Website) that tells the reader what sort of evidence this is before reading any further.

2. Title

```
Estate of Peter Vogel, also known as Robert Lance Vogel
```

The name the source gives itself, in bold. It is never paraphrased, so the reader can search for it exactly as written.

3. Provenance

```
Kings County Surrogate's Court, Brooklyn · file 2023-2443
```

In italics: who made the source, when, its file or catalog number, and its license where that matters (public domain, Creative Commons, copyrighted). This line lets the reader judge the source's authority, and find it again if the link ever dies.

4. Preview

`The estate file opened in April 2023 …`

A plain summary of what the reader would actually find on arriving there, sometimes with a small screenshot or key figures beside it. It is written from the source, not from the page's argument, so the reader can check whether the source really says what the page claims.

5. Why it is cited

`the only public record of Vogel's death that has been found`

The job the source does on this page: it proves a fact, supplies a date, or presents the other side. Together with the preview, this is what a bare footnote never gives.

6. The link

`→ websurrogates.nycourts.gov · Kings County, file 2023-2443`

Always last, so the reader knows what the source is before being invited to leave the page. The visible text shows the real web address, never a disguised one. A link later found to be dead is marked as broken rather than silently removed.

The markup

Each card is one `<article class="ref" id="r15">` element in column B. Inside it, in order: `span.rnum` (the number), `span.rkind` (the kind), `div.rtitle`, `div.rmeta` (provenance), `div.rprev` (preview, which may hold an `img.rshot` thumbnail or a `.rfacts` line of figures), `div.rwhy`, and `a.rlink` (the link, with a CSS arrow added before it).

A citation numeral in columns A or C is `<span class="fn" data-ref="r15">15</span>`. A small script finds the card whose `id` matches `data-ref`, switches to column B on a phone, scrolls the card to the middle of the screen, and adds a `flash` class for a moment to outline it. Nothing about this needs a network: the numeral and the card are in the same file.

How the previews reach the internet

Short answer: they don't, at reading time. A GYR-O page is a static file served by GitHub Pages. There is no server program behind it, and while a reader is reading, the page requests nothing from any other website. Everything a preview shows is gathered once, at build time, by the author's tools, and written into the page itself. The browser would block most live lookups anyway: its same-origin rule (CORS) stops a page from reading another site's content unless that site opts in.

Gathering at build time has four advantages. The page works with the network off. An archived copy still shows every preview years later. Readers' addresses are never sent to anyone else's server logs. And with no live connections there is nothing to hijack. The cost is that every preview is a snapshot, so it records the date it was taken.

Four separate mechanisms do the gathering.

1. The written preview (part 4)

Written by Claude while building the page. During research Claude opens each source with its web search and page-fetch tools, reads it, and writes the summary. The result is ordinary HTML text inside `div.rprev`. The page holds the words, not a connection.

2. Thumbnails: bake_card_media.py

Where a source is a picture, a scan or a PDF, a small image helps more than a sentence. This Python script in *Three Columns Tools/card-media/* walks a page's cards, downloads each eligible image once (Python's `urllib`, identifying itself as `ThreeColumns-media/1.0`), shrinks it to about 220 pixels wide, saves it in the site's `shots/` folder, and inserts it into the preview.

It keeps three rules. **Baked:** the reader's browser loads the local copy and never contacts the far server. **Licensed:** it refuses unless the provenance line (part 3) says reproduction is allowed: public domain, Creative Commons, open license, government work, no known copyright. Anything else keeps its sentence. `--force` overrides for a card you have cleared yourself. **Additive:** the image never replaces the sentence, and carries `alt=""` so a screen reader doesn't read the same thing twice.

3. Wikipedia term previews: bake_wiki_previews.py

Not part of a card, but the same idea applied inline. A term in the text marked `<span class="wk" data-wk="Page_title">` gets a small hover card. *BAKE WIKI PREVIEWS.command* calls Wikipedia's public summary service (`en.wikipedia.org/api/rest_v1/page/summary/`*title*) once per term and stores each extract in a JSON block inside the page (`<script type="application/json" id="wk-data">`). `wiki-preview.js` reads that block when a reader hovers or taps. Each card shows the date its text was taken and carries the CC BY-SA credit that Wikipedia's license requires. A term with no stored extract stays plain text.

4. Link health: CHECK LINKS.command

Run on your Mac, not by the reader. It finds every GYR-O site on the Archive volume and every `a.rlink` address in each page. It asks each address for a response (a `HEAD` request, falling back to `GET` where a server refuses HEAD; 25-second timeout), then writes the results to that site's own `links.json`. When a reader opens the page, the one request it makes is for that `links.json`, from its own site, uncached. Any card whose link is listed as dead gets the `broken` class and is shown as broken. If the file is missing, the page makes no claim either way.

Also: the internal card

Above the numbered cards sits one card about the page itself: who wrote it, when it last changed, and what changed. It is the page's own provenance line.
